

CAST: Testing the Attribution Stability of AI-Generated-Code Detectors Under Controlled Source Transformations

Anonymous Author(s)
Submission to ICSE 2027 NIER Track

Abstract—AI-generated-code detectors are evaluated almost exclusively on static snapshots, even though real code is routinely reformatted, renamed, and restructured after creation. We present CAST (Code Attribution Stability Testing), a controlled, distance-instrumented methodology that measures whether a detector’s human-versus-AI attribution survives such changes, and uses a canonicalization intervention to diagnose the source of any instability. Evaluating three independently reproducible detectors on 199 held-out Python and Java programs from the Droid corpus, we find that reformatting with a standard tool (`black/google-java-format`) flips DroidDetect-Base’s correct verdict on genuinely human-written code to “AI” 58.0% of the time (29/50 baseline-correct human samples), while leaving its verdict on actually AI-authored code almost unchanged (0.0–4.3%) – a class-specific, false-accusation-dominated pattern, not explained by human code needing larger edits (flipped human samples have *smaller* mean edit distance than unflipped ones). LLMsNiffer’s low error rate is not evidence of robustness: its own accuracy on this corpus (67.0% Python, 44.4% Java) is at or below a majority-class baseline. DetectCodeGPT shows broad score movement that a fresh-seed identity control shows cannot yet be distinguished from its own re-scoring noise. Canonicalizing away presentation differences attenuates DroidDetect-Base’s residual formatting-induced movement by 14.8–15.4 $\times$, though this is complicated by canonicalized input itself becoming out-of-distribution for the detector. Static accuracy alone therefore does not characterize detector reliability under routine code evolution.

Index Terms—AI-generated code detection, robustness testing, metamorphic-style evaluation, software evolution

I. INTRODUCTION

AI coding assistants now write a substantial share of the code developers review, edit, and ship. Detectors that attribute this code to a human or AI origin are typically evaluated the same way a classifier is: on a fixed test set, scored once, with accuracy as the verdict.

Motivation: That protocol quietly assumes the code being judged stays put. It does not. An IDE reformats a file on save, a refactor renames a variable, a reviewer rewrites a loop – none of it changes what the program does, yet any of it can change what a detector says about who wrote it. A detector whose verdict flips under a cosmetic edit is not measuring origin; it is measuring the snapshot. This is a routine, non-adversarial threat distinct from the adversarial setting the closest prior work targets: robustness studies vary

how code is generated [11] or train against an attacker’s renaming [6], [1], and separate work shows renaming can defeat code watermarks [7] and that code-model metamorphic testing treats renaming as standard [8]. None measures whether an already-trained detector’s verdict survives a non-adversarial edit, or reports which edits matter and why.

Objective: This paper’s objective is to make attribution stability an explicit, measurable property of an AI-code detector, not an afterthought to accuracy, and to build the smallest instrument that can measure it. We introduce **CAST** (Code Attribution Stability Testing): given a program of known origin, CAST applies a validated transformation, measures the resulting change at the text, token, and AST level, and records how the detector’s score and decision move (Section II). Where movement is large, CAST adds a second, targeted step: canonicalizing both sides of the pair to test whether the instability is a presentation artifact or something deeper. Applied to three independently reproducible detectors over 199 held-out Python and Java programs [1], this approach exposes stability profiles that a single accuracy number cannot: one detector’s instability is almost entirely a formatting artifact that canonicalization removes; another’s is not.

This paper makes two contributions.

- **CAST**, a controlled, distance-instrumented diagnostic methodology for evaluating the attribution stability of already-trained AI-code detectors under routine, non-adversarial edits, including a canonicalization intervention that isolates presentation-driven instability from a broader failure mode.
- A held-out evaluation of three reproducible detectors over 199 programs, showing qualitatively different, transformation-specific stability profiles, together with a corpus-composition limitation that bounds this evaluation’s current reach.

The remainder of the paper is organized as follows. Section II defines CAST and the canonicalization intervention. Section III describes the dataset, transformations, detectors, and research questions. Section IV reports results; Section V discusses their implications. Section VI lists threats to validity, Section VII positions CAST against related work, Section VIII outlines future plans, and Section IX concludes.

II. CAST

A. Attribution stability

Attribution stability is the degree to which a detector’s output for a program is preserved under routine, non-adversarial edits – distinct from adversarial robustness, which studies a worst-case attacker searching for an evasive rewrite. CAST’s transformations are fixed, deterministic operators applied uniformly across the corpus, not an adversarial search against any detector. CAST treats score movement and decision-flip rate as first-class observations, not a single post-transformation accuracy number.

B. Methodology

For a detector D , a transformation T , and a program C_0 of known provenance, CAST proceeds in five steps.

- 1) Apply T to obtain $T(C_0)$.
- 2) Validate that $T(C_0)$ is well-formed at or above C_0 ’s own syntactic-validity tier. Non-applicable and invalid outcomes are recorded, not discarded.
- 3) Measure d_{ext} (character-level edit distance), d_{token} (edit distance over the language’s own lexical token stream, e.g., Python’s `tokenize` output including `INDENT/DEDENT`, or Java tree-sitter leaves), and d_{AST} (AST node-count change).
- 4) Score $D(C_0)$ and $D(T(C_0))$.
- 5) Aggregate, per detector, language, and family, the mean and median score movement $\Delta_D = D(T(C_0)) - D(C_0)$, the decision-flip rate, and the **Induced Error Rate**, defined as:

$$\text{IER} = \frac{\#\{\text{baseline-correct programs whose decision flips after } T\}}{\#\{\text{baseline-correct programs}\}} \quad (1)$$

IER is conditioned on the detector having been correct on C_0 . A flip on a program the detector already misclassified would otherwise conflate ordinary detector error with transformation-induced instability. Detectors also expose incompatible score semantics: a calibrated probability, an uncalibrated sigmoid output, or an unnormalized statistic. CAST therefore does not compare raw $|\Delta_D|$ across detectors. IER, a bounded proportion with a consistent interpretation regardless of score scale, is the primary cross-detector metric. Raw score movement is interpreted only within a given detector’s own distribution.

C. Canonicalization intervention

When a transformation family produces large score movement, CAST additionally scores $F(C_0)$ and $F(T(C_0))$ for a deterministic canonicalizer F that removes a specific, named category of variation (in this study: whitespace, blank lines, comments, and quote style, implemented by re-tokenizing the program and rejoining its semantic tokens). A pair for which $F(C_0)=F(T(C_0))$ collapses to $\Delta=0$ by construction. This subset is reported separately from the residual pairs for which a genuine difference remains after canonicalization, so that the diagnostic result is not an artifact of the trivially-resolved cases. A substantial reduction in the residual subset’s score movement constitutes intervention evidence that the removed

category of variation contributes to the observed instability. It does not constitute proof of a specific internal mechanism.

III. STUDY DESIGN

Dataset: This evaluation uses DroidCollection’s [1] held-out test split: 199 programs (100 Python, 99 Java), stratified across `HUMAN_GENERATED`, `MACHINE_GENERATED`, and `MACHINE_REFINED`.

Transformations: Three confirmatory families: formatting (`black/google-java-format`), lexical renaming of one identifier, and control-flow rewriting (one canonical loop-form rewrite), with applicability rate 0.616 and success rate 0.992 among applicable attempts. A fourth stage attempted two deeper families, local structural extraction and semantics-preserving rewriting, reported as a feasibility finding (Section IV-D), not a confirmatory one.

Validation: Python uses `compile()`-based validation. Java is tiered, given the corpus’s many function/class fragments: `STANDALONE_VALID` > `WRAPPER_VALID` (compiles inside a synthetic harness, used only as a validation instrument and never scored) > `STRUCTURAL_ONLY` > `PARSE_INVALID`.

Detectors: Three of five targeted detectors were independently reproducible from public artifacts. *LLMSniffer* [2] (GraphCodeBERT [19], supervised-contrastive) reaches 90.0% on GPTSniffer’s Java test subset [3] but only 67.0%/44.4% on this corpus (at or below majority-class baseline; Section VI). *DroidDetect-Base* (ModernBERT-base [18] with a projection layer and a four-class head) ships a checkpoint with no modeling code; we reconstructed its architecture and validated it against a disjoint dev-split subset ($n=24$), reaching 87.5% accuracy (Wilson 95% CI [69.0,95.7]) – consistent with, though less precise than, its 98–99% test-split accuracy (Section IV-A) given the small dev sample. *DetectCodeGPT* [4] is a zero-shot statistic from the DetectGPT [14]/NPR [15] family, computed from CodeLlama-7B [16] and CodeT5p-770M [17], with $K=50$ perturbations per a pre-registered convergence study. Its threshold, calibrated on an independent 40-sample set, reaches only 60.0% accuracy, so raw score movement is primary evidence and threshold-derived decisions secondary. *GPTSniffer* [3] and *CodeGPTSensor/+* [5] could not be integrated, as neither released a checkpoint; we did not train replacements, to avoid confounds distinct from evaluating a published detector.

RQs: This evaluation is structured around three research questions.

- **RQ1:** How stable are detector scores and decisions under formatting, lexical renaming, and control-flow rewriting?
- **RQ2:** Do detectors exhibit similar or detector-specific stability profiles under identical transformations?
- **RQ3:** Does canonical normalization attenuate transformation-associated instability, and does this differ across detectors?

TABLE I
DROIDDETECT-BASE FORMATTING IER BY TRUE ORIGIN CLASS
(HUMAN-VS-AI COLLAPSE, POOLED ACROSS LANGUAGES).

True class	n (baseline-correct)	IER	95% CI
HUMAN_GENERATED	50	58.0% (29/50)	[44.2,70.6]
MACHINE_GENERATED	58	0.0% (0/58)	[0.0,6.2]
MACHINE_REFINED	47	4.3% (2/47)	[1.2,14.3]

TABLE II
DROIDDETECT-BASE INDUCED ERROR RATE BY LANGUAGE AND FAMILY
(n = BASELINE-CORRECT DENOMINATOR, HUMAN-VS-AI COLLAPSE, SECTION VI). LLMSNIFFER (IER $\leq 8.3\%$ THROUGHOUT, BUT AT/BELOW MAJORITY-CLASS BASELINE) AND DETECTCODEGPT (IER 16.7–50.0%, WITHIN ITS 30.0% NOISE FLOOR) ARE SUMMARIZED IN TEXT; FULL PER-FAMILY TABLES ARE IN THE ARTIFACT.

Detector	Lang.	Family	IER (n)	95% CI
DroidDetect-Base	Py	formatting	26.1% (24/92)	[18.2,35.9]
DroidDetect-Base	Java	formatting	11.1% (7/63)	[5.5,21.2]
DroidDetect-Base	Py	lexical rename	0.0% (0/80)	[0.0,4.6]
DroidDetect-Base	Java	lexical rename	1.2% (1/82)	[0.2,6.6]
DroidDetect-Base	Py	control flow	0.0% (0/26)	[0.0,12.9]
DroidDetect-Base	Java	control flow	6.2% (1/16)	[1.1,28.3]

IV. RESULTS

A. RQ1/RQ2: detector-specific, transformation-specific profiles

Table II reports IER with baseline-correct denominators. DroidDetect-Base’s formatting IER exceeds its lexical-rename and control-flow IER in both languages, collapsing to at or near 0% in Python; Wilson [20] 95% CIs for formatting do not overlap the other families in Python, while Java’s smaller denominators give wider, overlapping intervals. Table I breaks this down by true origin class: DroidDetect-Base’s formatting IER is 58.0% on genuinely human-written code but 0.0–4.3% on actually AI-authored code, in both languages separately (Python 66.7%/0.0%/6.5%; Java 41.2%/0.0%/0.0%). This is not a no-op artifact (only 5–6% of AI-class transformations are byte-identical); nor is it explained by edit size – flipped human samples have *smaller* mean d_{text} (81.1) than unflipped ones (470.5). $|\Delta|$ correlates weakly with d_{text} ($\rho \leq 0.30$) but not d_{AST} ($\rho = -0.20, 0.03$), consistent with a presentation- not structure-driven effect. LLMSniffer’s low IER is not robustness evidence (67.0%/44.4% baseline accuracy, at/below majority; Section VI); DetectCodeGPT’s elevated IER (16.7–50.0%) sits within its 30.0% identity-control noise floor (Section VI).

Answer: DroidDetect-Base’s instability is real, class-specific, and overwhelmingly false-accusing; the other two detectors’ apparent profiles are confounded by degenerate baseline accuracy and re-scoring noise, respectively.

B. RQ3: canonicalization intervention

The canonicalization diagnostic was applied to all 158 formatting pairs (94 Python, 64 Java). Canonicalizing both sides collapsed **108/158 (68.4%)** to byte-identical input ($\Delta=0$ by construction); the remaining **50/158** retained a residual

TABLE III
MEAN $|\Delta|$ BEFORE/AFTER CANONICALIZATION, BY LANGUAGE (NOT COMPARABLE ACROSS DETECTORS).

Detector	Lang.	Full set		Residual-only	
		before	after (red.)	before	after (red.)
LLMSniffer	Py (94/32)	0.027	0.003 (8.6 $\times$)	0.032	0.009 (3.5 $\times$)
LLMSniffer	Java (64/18)	0.025	0.002 (16.7 $\times$)	0.020	0.005 (3.7 $\times$)
DroidDetect-Base	Py (94/32)	0.315	0.009 (34.9$\times$)	0.394	0.027 (14.8$\times$)
DroidDetect-Base	Java (64/18)	0.161	0.003 (63.3$\times$)	0.139	0.009 (15.4$\times$)
DetectCodeGPT	Py (94/32)	0.687	0.280 (2.5 $\times$)	0.843	0.824 (1.0 $\times$)
DetectCodeGPT	Java (64/18)	0.645	0.244 (2.6 $\times$)	0.566	0.866 (0.7$\times$, increase)

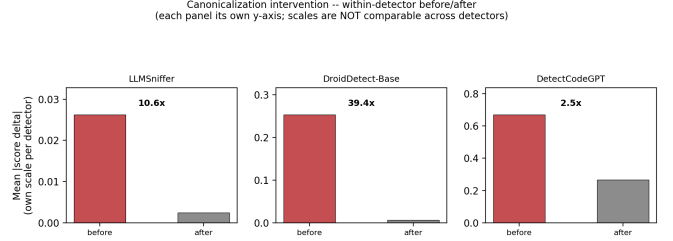


Fig. 1. Canonicalization reduction factor, per detector (not cross-comparable).

difference and were rescored. Table III reports both, isolating the diagnostic’s contribution from the trivially-resolved cases.

On the residual-only subset – which excludes pairs canonicalization trivially equalizes – DroidDetect-Base’s mean score movement still falls 14.8–15.4 $\times$, but this is complicated by an out-of-distribution effect: on the 100 canonicalized residual scores, its mean probability on the *true* class drops to 0.223 (66% < 0.05) against 98–99% raw baseline accuracy (Section VI), so canonicalized input is itself often confidently misclassified. DetectCodeGPT shows no comparable reduction (Python 1.0 $\times$, Java 0.7 $\times$ increase) and its canonicalized scores skew toward the AI decision (68% above threshold, vs. 50–54% baseline) – a further distributional shift. LLMSniffer shows a smaller, real reduction with no comparable saturation. Pooling both languages (Fig. 1) gives full-set factors of 10.6 $\times$ /39.4 $\times$ /2.5 $\times$; a ratio from rounded pooled means, rather than the underlying values, would overstate these by 2–3 $\times$, hence the language-level reporting here.

Cohen’s d_z [21] is reported in the accompanying artifact but not used as a headline statistic, since it standardizes by variance and can appear large even for a practically small movement (e.g., LLMSniffer, $d_z=1.39$ –1.49 despite mean movement below 0.025).

Answer: Canonicalization sharply reduces DroidDetect-Base’s residual movement, though part of this may reflect canonicalized input becoming out-of-distribution rather than purely resolved instability; DetectCodeGPT’s movement is unreduced, and on Java, larger after canonicalization.

C. Corpus feasibility for deeper transformations

Local structural extraction and semantics-preserving rewriting were additionally attempted at two intensities; of 796 attempted combinations, only 37 (4.65%) were applicable (35 validated, only 4 with true differential-execution verification), reflecting Droid’s fragment-heavy composition. The resulting

per-detector cells ($n=1-12$) are illustrative only and support no confirmatory claim.

V. DISCUSSION

Static accuracy does not characterize post-transformation reliability, but neither does a low IER by itself characterize stability: LLMSniffer’s near-majority-class accuracy makes its low IER a likely degenerate-predictor artifact, and DetectCodeGPT’s IER is not yet separated from re-scoring noise. Only DroidDetect-Base’s instability survives both checks – a real detector (98–99% baseline accuracy) whose formatting sensitivity is reproducible and overwhelmingly false-accusing. A verdict that depends on a random seed, or on a detector no better than guessing the majority class, is unfit for single-verdict use. Table I rules out the simplest confound – formatting as a near-no-op on AI code – since flipped human samples have *smaller* edits than unflipped ones; a detector that learned tidy formatting as an AI-authorship proxy fits better than edit magnitude, though CAST does not identify the mechanism directly. Canonicalization is a diagnostic, not a validated mitigation: its own out-of-distribution effect means a practical fix needs a layout-preserving normalizer, not this token-rejoining canonicalizer.

VI. THREATS TO VALIDITY

- **External:** The evaluation covers two languages, one corpus, and three detectors; two others lacked a released checkpoint. The transformation operators represent common tooling edits, not sampled developer histories.
- **Construct:** LLMSniffer’s baseline accuracy (67.0%/44.4%, at/below majority) means its low IER is not robustness evidence. Detector scores have incompatible scales, and DetectCodeGPT’s weak calibration limits threshold-derived results to secondary status; its $IER/|\Delta|$ are upper bounds including a comparably sized identity-control noise floor. d_{text} , d_{token} , d_{AST} correlate only weakly with $|\Delta|$.
- **Internal:** DroidDetect-Base’s architecture was reconstructed absent public modeling code, not yet compared against the original authors’ numbers. Its IER uses the human-versus-AI collapse of its four-class output; 16 of 365 flips were intra-AI churn, excluded on this basis. The canonicalizer normalizes several presentation properties at once, so a result is about that combined category, not which is responsible; canonicalized-input accuracy shows a real out-of-distribution effect for DroidDetect-Base not fully controlled for.
- **Corpus:** Droid’s fragment-heavy composition limits deeper transformations to 4.65% applicability and differential-execution verification to 4/35 successful instances; those cells are illustrative, not confirmatory.
- **Conclusion:** Some cells have small denominators, as low as $n=4$; Wilson 95% CIs are reported throughout. Cohen’s d_z is supplementary, directional-consistency evidence only.

TABLE IV
NOVELTY POSITIONING AGAINST THE CLOSEST PRIOR WORK.

Work	Post-gen. edit	Routine, non-adv.	AI-code detector	Cross-det.
Detector papers [3], [2], [4], [5]	partial	partial	✓	–
CodeGPTSensor+ [6], Droid [1]	✓	– (adversarial)	✓	partial
Pordanesh et al. [11]	–	–	✓	✓
RoPGen [12]	✓	✓	– (human authorship)	–
Code-model robustness [9], [10], [7], [8]	✓	✓	–	partial
CAST (this work)	✓	✓	✓	✓

VII. RELATED WORK

Measuring prediction change under fixed, semantics-preserving transformations is established for code models generally: Rabin et al. [9] show neural program models fail to generalize under such edits, and ReCode [10] benchmarks code-generation robustness under >30 similar operators; CAST applies the same kind of edit to a different task and failure mode – attribution false-accusation, not degraded generation. Table IV positions CAST against the closest prior work on whether it edits code post-generation, is routine rather than adversarial, targets already-trained AI-code detectors specifically, and compares detectors under identical edits. Detector papers [3], [2], [4], [5] report accuracy, occasionally under one perturbation, without stratifying transformations. CodeGPTSensor+ [6] and Droid [1] share the observation that renaming can obscure AI origin, but address it with adversarial training or attack scenarios. Pordanesh et al. [11] vary training-set and prompting conditions, not the source code itself. RoPGen [12] applies style transformation *during training* to harden a *human*-authorship attributor; CAST transforms *at evaluation time*, against *already-trained*, unmodified detectors. Whitespace-based detection work [13] motivates why formatting sensitivity can be presentation-driven.

VIII. FUTURE PLANS

This evaluation will be extended in three directions. (1) A symmetric experiment perturbing AI code toward human-like “messiness” to test for the reverse flip, directly probing the tidy-formatting-as-AI-proxy hypothesis, plus a layout-preserving normalizer as a genuine mitigation candidate. (2) A corpus of complete, executable programs with chained transformations mined from real commit histories, beyond the 4.65% applicability of function-level fragments. (3) Comparing reconstructed DroidDetect-Base against the original authors’ numbers, shared-seed DetectCodeGPT, and a detector trained against CAST’s transformations.

IX. CONCLUSION

This paper presented CAST, a controlled, distance-instrumented methodology for testing attribution stability. Its clearest finding: a validated detector flips correct verdicts on human-written code to “AI” 58.0% of the time after routine reformatting, versus 0.0–4.3% on actually AI-authored code – consistent with tidy formatting as a learned AI-authorship proxy, not edit magnitude. The other two detectors’ profiles are confounded by baseline accuracy and re-scoring noise: a low IER alone is not robustness evidence.

REFERENCES

- [1] D. Orel, I. Paul, I. Gurevych, P. Nakov, “Droid: A Resource Suite for AI-Generated Code Detection,” *EMNLP*, 2025.
- [2] M. L. Dihan, A. Muhtasim, “LLMSniffer: Detecting LLM-Generated Code via GraphCodeBERT and Supervised Contrastive Learning,” arXiv:2604.16058, 2026.
- [3] P. T. Nguyen, J. Di Rocco, C. Di Sipio, R. Rubel, D. Di Ruscio, M. Di Penta, “GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT,” *Journal of Systems and Software*, 2024.
- [4] Y. Shi, H. Zhang, C. Wan, X. Gu, “Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers,” *ICSE*, 2025.
- [5] X. Xu, C. Ni, X. Guo, S. Liu, X. Wang, K. Liu, X. Yang, “Distinguishing LLM-generated from Human-written Code by Contrastive Learning,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 91, 2025.
- [6] X. Yin, X. Li, C. Ni, X. Xu, X. Yang, “Detecting LLM-generated Code with Subtle Modification by Adversarial Training,” arXiv:2507.13123, 2025.
- [7] T. Suresh, S. Ugare, G. Singh, S. Misailovic, “Is the Watermarking of LLM-Generated Code Robust?” arXiv:2403.17983, 2024.
- [8] A. Asgari, M. de Koning, P. Derakhshanfar, A. Panichella, “Metamorphic Testing of Deep Code Models: A Systematic Literature Review,” arXiv:2507.22610, 2025.
- [9] M. R. I. Rabin, N. D. Q. Bui, K. Wang, Y. Yu, L. Jiang, M. A. Alipour, “On the Generalizability of Neural Program Models with respect to Semantic-Preserving Program Transformations,” *Journal of Systems and Software*, 2021.
- [10] S. Wang, Z. Li, H. Qian, C. Yang, Z. Wang, M. Shang, V. Kumar, S. Tan, B. Ray, P. Bhatia, R. Nallapati, M. K. Ramanathan, D. Roth, B. Xiang, “ReCode: Robustness Evaluation of Code Generation Models,” *ACL*, 2023.
- [11] S. Pordanesh, S. Bukhari, B. Tan, L. De Carli, “Hiding in Plain Sight: On the Robustness of AI-Generated Code Detection,” *DIMVA*, 2025.
- [12] Z. Li, G. Q. Chen, C. Chen, Y. Zou, S. Xu, “RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation,” *ICSE*, 2022.
- [13] S. M. H. Nirob, S. Ehsan, M. Rahman, S. Haque, “Whitespaces Don’t Lie: Feature-Driven and Embedding-Based Approaches for Detecting Machine-Generated Code,” arXiv:2601.19264, 2026.
- [14] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, C. Finn, “DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature,” arXiv:2301.11305, 2023.
- [15] J. Su, T. Y. Zhuo, D. Wang, P. Nakov, “DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text,” *Findings of EMNLP*, 2023.
- [16] B. Rozière, J. Gehring, F. Gloeckle, et al., “Code Llama: Open Foundation Models for Code,” arXiv:2308.12950, 2023.
- [17] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, S. C. H. Hoi, “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” arXiv:2305.07922, 2023.
- [18] B. Warner, A. Chaffin, B. Clavié, et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” arXiv:2412.13663, 2024.
- [19] D. Guo, S. Ren, S. Lu, et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” arXiv:2009.08366, 2020.
- [20] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [21] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.